

AUTOMATION TESTING ON BESTBUY WEB APPLICATION

Selenium

Pytest

BDD

Behave

Domain :- E-Commerce

Wipro Talent Next | Capstone Project | 2026

TEAM MEMBERS

NAME	Shaurya Singh	Saumy Singh	Arpit Das	Ayush Raj	Pankaj Rana	Ashish Kumar
	Trending Deals	Deal of the Day	Shop by Brand (Apple)	TV & Home Theater	Product Search	Headphones

BestBuy Automation

Deal of the Day · Capstone Project

Saumy Singh | 4959462 | Deal of Day.

FRAMEWORK LAYERS

TEST

test_end_to_end.py | test_positive_negative_tc.py

PAGES

HomePage | DealOfDayPage | CartPage | CheckoutPage

UTILS

LogGenerator | ScreenshotUtil | CSVReader

CONFIG

conftest.py | test_data.csv

BDD

environment.py | Feature Files + Steps

Selenium · Pytest · Behave BDD · Allure · POM · Edge WebDriver

7

Test Scenarios

5

Positive Cases

2

Negative Cases

✓ POSITIVE TEST SCENARIOS



End-to-End: Homepage → Deal → Cart → Checkout with email



Open Deal of the Day page — verify URL & CSS indicator



Select first product → verify product details page loads



Add Deal product to cart → verify cart not empty



Proceed to Checkout → verify checkout page loads

✗ NEGATIVE TEST CASES



Invalid Email at Checkout — malformed email → validation error shown



Checkout Without Product — cart empty validation

KEY METHODS — DealOfDayPage

is_deal_page_displayed()

URL + CSS indicator check

wait_and_add_to_cart()

Multi-selector, scroll, JS click fallback

go_to_cart()

Link/button fallback + URL guard

AUTOMATION WORKFLOW

1

Read CSV

Load emails

2

Open Browser

BestBuy home

3

Navigate

Deal of the Day

4

Add to Cart

JS click

5

Checkout

Enter email

6

Allure Report

HTML + screenshots

Best Buy Automation

Trending Deals · Capstone Project

Shaurya Singh | 4957864 | Trending Deals

FRAMEWORK LAYERS

TEST

conftest.py | pytest tests

PAGES

BasePage | CategoryPage | HomePage

UTILS

LogGenerator | ScreenshotUtil | CSVReader | driver | JSONReader | CSVReader

BDD

environment.py | Feature Files + Steps

Selenium · Pytest · Behave BDD · Allure · POM · ChromeDriver

7

Test Scenarios

3

Positive Cases

4

Negative Cases

✓ POSITIVE TEST SCENARIOS



Selecting Category — category name loaded from JSON file, navigates to category page



Price Range Validation (100 to 1000) — products visible, first price within range



Price Range Validation (500 to 2000) — products visible, first price within range

END-TO-END Tests

Home Page(Main page of the Website) --> Base Page(Trending Deals Page) -> Category Page(Laptop & Desktop Page) --> Cart Page

✗ NEGATIVE TEST CASES



Invalid Range Alert (500,100) — min > max triggers browser alert



Invalid Range Alert (100, 0) — min > max triggers browser alert



Zero Range Grid Vanishes (0,0) — product grid empties after filter



Zero Range Grid Vanishes (0,100) — product grid empties after filter

KEY PAGE — CategoryPage

Applies price filters (positive, invalid range, zero range)

Handles alert assertions and product grid validation

Validates first product price against the filter range

Uses XPath to target first product-grid and price elements

AUTOMATION WORKFLOW

1

Open Browser

Through Driver

2

Apply Filter

Min / Max price set

3

Validate

Products + price check

4

Add to Cart

Clicking on cart icon

5

Cart Icon

Navigate to cart

6

Allure Report

Screenshots + logs

BestBuy Website Automation Framework (shop by brand [Apple])

★ ArpitDas_4958513_WiproCapstone

Selenium • Pytest • Behave BDD | Python Automation | Wipro Talent Next — Arpit Das (4958513)

⚙️ FRAMEWORK ARCHITECTURE

Framework (POM + Selenium Pytest/BDD Behave + Data-Driven)

Page Object Model — HomePage · MacBookPage · CartPage
Excel-Driven Parametrized Testing (openpyxl)
Selenium Pytest test files
BDD Gherkin Feature Files + Behave Runner
Utility Layer — Logger · Screenshot · Config · ExcelUtils

✅ FEATURES AUTOMATED

- Country Selection + Homepage Validation
- Top Deals → Apple Section → MacBook Listing
- Processor Filter Expansion & Selection
- Add to Cart + Cart Item Verification
- Checkout Navigation + Email Field Validation
- End-to-End Complete Purchase Workflow

🔧 TECHNOLOGIES USED

Python 3.x

Selenium WebDriver

Pytest

Behave BDD

Allure Reports

OpenPyXL / GitHub

📝 TEST CASES

✓ POSITIVE TEST CASES

- TC_01
- Apple M4 — Add to Cart Flow
-
- TC_02
- Apple M3 — Add to Cart Flow
-
- TC_03
- Apple M4 — Checkout Navigation
-
- TC_04
- Apple M3 — Cart Validation
-
- TC_08
- Apple Section Navigation Flow

✗ NEGATIVE TEST CASES

- TC_05
- Invalid Processor — Filter Validation
-
- TC_06
- Unsupported Processor — No Results
-
- TC_07
- Invalid Email at Checkout Page

📊 REPORTING & UTILITIES

- ▶ Allure HTML

Auto-generated after every run
- ▶ Screenshots

Every step + on-failure capture
- ▶ LogGen Logger

automation.log with timestamps
- ▶ Excel Data

4 sheets —
Positive/Negative/Checkout/Nav
- ▶ Failure Track

Categories + Timeline view in Allure

⚡ WORKFLOW:

Feature File and Step Def./
Pytest Test Cases

environment.py /
confest.py

Page Objects
(POM)

Selenium
Actions

Allure
Report

9

Total Tests

5

Positive TCs

4

Negative TCs

100%

Pass Rate

2

Frameworks

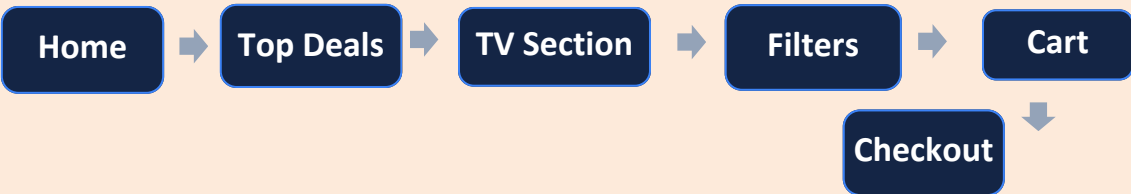
11

Automated Steps

TEST SCENARIOS 9 Total

END-TO-END WORKFLOW 1 Scenario

Complete TV Purchase: Navigation to Checkout
File: test_bestbuy_workflow.py



POSITIVE TEST CASES 5 Scenarios

File: test_positive_negative_tc.py

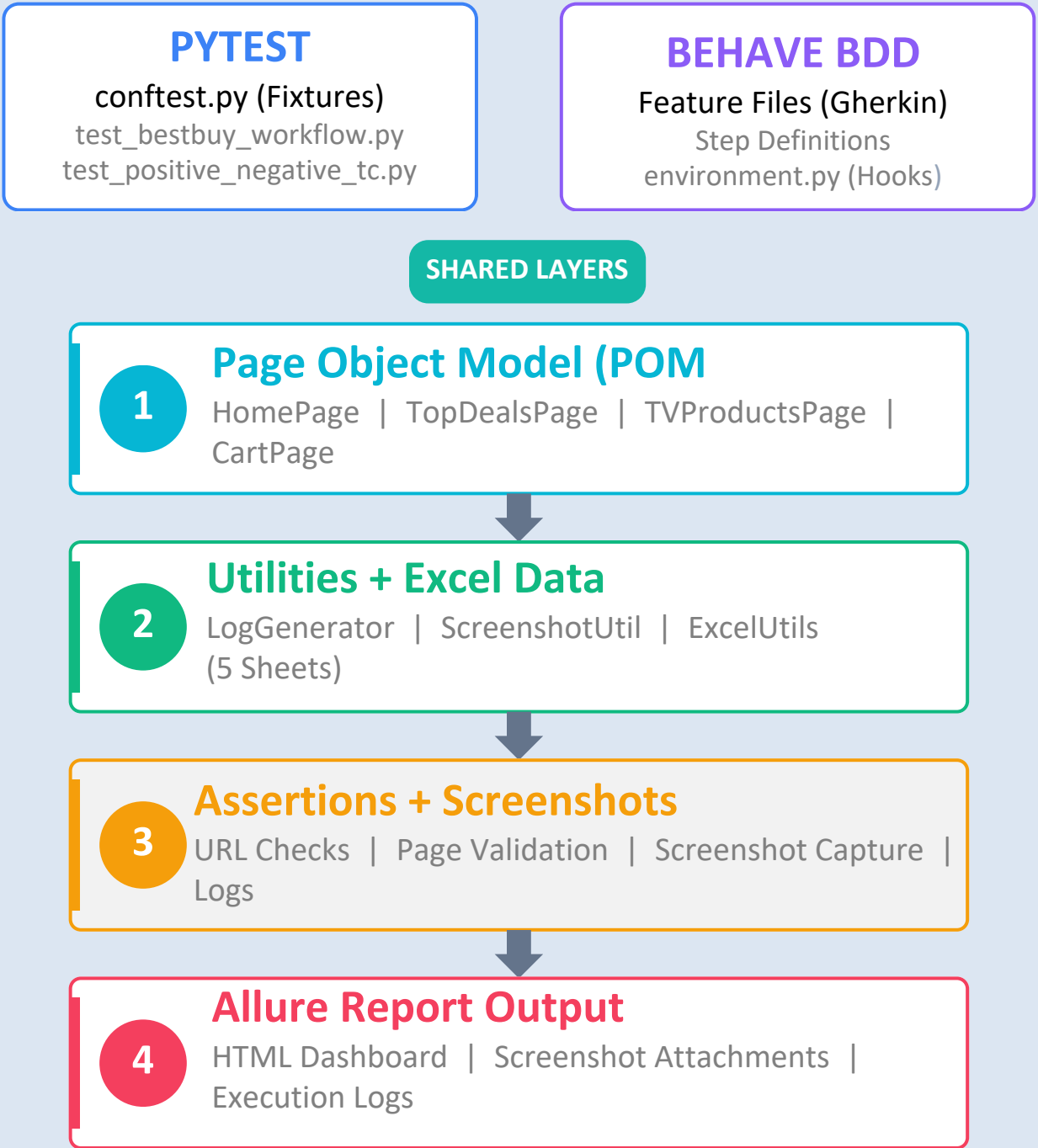
- + Top Deals Page Navigation
- + TV & Home Theater Section Navigation
- + Brand Filter Application (Excel Data-Driven)
- + Price Range Filter Validation
- + Product Quantity Update in Cart

NEGATIVE TEST CASES 3 Scenarios

File: test_positive_negative_tc.py

- x Invalid Price Range Filter (Reversed/Extreme)
- x Invalid Brand Search (Non-Existent Brand)
- x Invalid Email at Checkout (Malformed Email)

HYBRID AUTOMATION FRAMEWORK WORKFLOW



TECHNOLOGIES USED

Python 3.x
Programming Language

Selenium
Browser Automation (Edge)

Pytest
Test Runner + Fixtures

Behave BDD
Gherkin + Step Definitions

Allure
Interactive HTML Reports

OpenPyXL
Excel Data-Driven Testing

PyCharm
Development IDE

Git / GitHub
Version Control

7

Test Scenarios

5

Positive Cases

2

Negative Cases

1

E2E Journey

FRAMEWORK LAYERS

TEST

test_search.py
test_e2e_search_journey.py

PAGES

HomePage · SearchPage
CartPage · LoginPage

UTILS

Logger · Screenshot
DriverFactory

CONFIG

conftest.py · test_data.json

BDD

search.feature
search_steps.py · environment.py

TEST SCENARIOS

✓ Positive

● TC_POS_01

Valid product search returns results

● TC_POS_02

Result titles match query keyword (≥30%)

● TC_POS_03

Search bar retains typed query

● TC_POS_04

Gibberish query handled gracefully

● TC_POS_05

E2E: Login → Search → Add to Cart

✗ Negative

■ TC_NEG_01

Invalid login — error shown

■ TC_NEG_02

Special characters handled gracefully

↻ E2E Journey: Login → Search → Verify → Add to Cart

KEY METHODS

search_for()

Type query, submit via Enter, wait for URL update and result items to load

add_first_available_to_cart()

Skips sold-out items, JS-clicks first enabled Add to Cart button

go_to_next_page()

Pagination using aria-label selector with URL guard for page confirmation

has_no_results_message()

Checks CSS + XPath selectors for empty-results state

PROJECT 1

Selenium 4 + PyTest + WebDriverManager + Allure

Framework & Architecture

Tech Stack: Selenium 4 · PyTest · WebDriverManager · Allure

Architecture: Page Object Model (POM) — Separate page class per screen, test logic kept clean

Key Libraries: WebDriverManager · Allure · pytest-html · OpenPyXL

Folder Structure (what each folder does)

config/ qa.yaml (base_url, browser, waits) · config.py (PyYAML loader)

pages/ POM classes — BasePage · HomePage · HeadphonesPage

tests/ E2E (1) · Positive (7) · Negative (3) test cases

utilities/ logger.py · screenshot_utils.py · excel_utils.py

test_data/ headphone_categories.xlsx — data-driven via OpenPyXL

reports/ Allure + pytest-html reports (auto-generated)

screenshots/ timestamped PNG per test step

logs/ one log file per run — saves all test activity

conftest.py launches and closes Chrome browser for each test

pytest.ini sets test paths and report output settings

Test Coverage (11 Scenarios)

1 E2E test_1_e2e — Full flow TC01→TC07 in ONE browser session

7 +VE TC01–TC07: Open Site → Country Select → Top Deals → Headphones → Sub-Cat → Add Cart → Cart Verify

3 -VE NC01 Wrong URL · NC02 Wrong Country · NC03 Wrong Section

MODULE FLOW

TEST JOURNEY

Selenium WebDriver E2E flow on bestbuy.com

TC01	Open BestBuy Website Opens bestbuy.com and confirms the correct page has loaded
TC02	Select United States Waits for the country popup, selects United States, lands on US homepage
TC03	Click Top Deals Hovers over menu, clicks Top Deals, confirms URL contains 'top-deals'
TC04	Click Headphones Category Finds Headphones link using XPath and clicks it via JavaScript
TC05	Select Random Sub-Category Reads sub-categories from Excel file, picks one randomly and clicks it
TC06	Add Products to Cart Finds active Add to Cart buttons, clicks 2 products, dismisses popup, cart shows 2
TC07	Open Cart & Verify Summary Opens the cart page and confirms products were added successfully

TECH EXECUTION

Behind the scenes on every test run

①	Config Loaded — reads browser settings (URL, wait time, headless mode) from config.py
②	Browser Started — WebDriverManager auto-downloads ChromeDriver and opens Chrome
③	Page Objects Used — each page has its own class; tests call actions like click, verify
④	Logging Active — every test action is recorded in a log file for debugging
⑤	Screenshot Saved — a PNG image is captured after each test step as proof
⑥	Reports Generated — Allure and HTML reports created automatically after the run

PROJECT 2

Selenium 4 + Behave BDD + WebDriverManager + Allure

Framework & Architecture

Tech Stack: Selenium 4 · Behave BDD · WebDriverManager · Allure

Architecture: Page Object Model (POM) — Same page classes reused from Project 1

Key Libraries: Gherkin Syntax · allure-behave · behave-html-formatter

Gherkin Sample (how test is written in BDD)

Scenario: TC03 — Click Top Deals
Given I open the BestBuy website
When I select United States
And I click Top Deals
Then the Top Deals page should open

Folder Structure (what each folder does)

config/ qa.yaml (base_url, browser, waits) · config.py (PyYAML loader)

pages/ BasePage · HomePage · HeadphonesPage (shared with P1)

features/ e2e · positive_cases · negative_cases (.feature / Gherkin)

steps/ common_steps · e2e · positive · negative (Python step defs)

environ. setup/teardown hooks (same role as conftest.py in PyTest)

utilities/ logger · screenshot_utils · excel_utils (shared with P1)

reports/ allure_reports/ + behave-html report (auto-generated)

run_tests.py clears old results, runs all BDD tests, generates the Allure report

behave.ini configures test paths and output settings

3 Utility Modules (shared by both projects)

logger.py timestamped .log per run — INFO+ · file + console handlers

screenshot_utils.py PNG per step — filename: test_name + timestamp

excel_utils.py Reads .xlsx via openpyxl → random.choice() picks category

Test Coverage (11 Scenarios)

1 E2E

7 +VE

3 -VE

Gherkin: Each step (Given/When/Then) maps to a Python function in the steps/ folder

DELIVERABLES

What we built and tested



AUTOMATION FRAMEWORK

Selenium WebDriver + PyTest + Behave BDD with Page Object Model (POM) architecture



TEST COVERAGE

End-to-End, Positive & Negative test cases across all team modules (100% pass rate)



ALLURE REPORTS

Professional HTML reports with step-by-step execution flow, screenshots & history



SCREENSHOT EVIDENCE

Automated screenshot capture at each key execution stage for audit & debugging



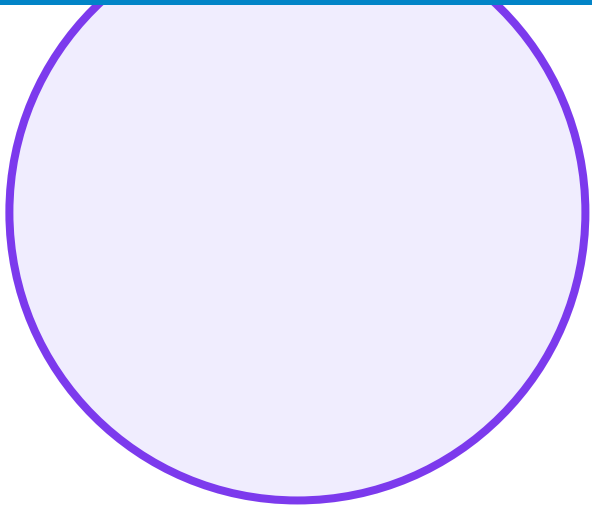
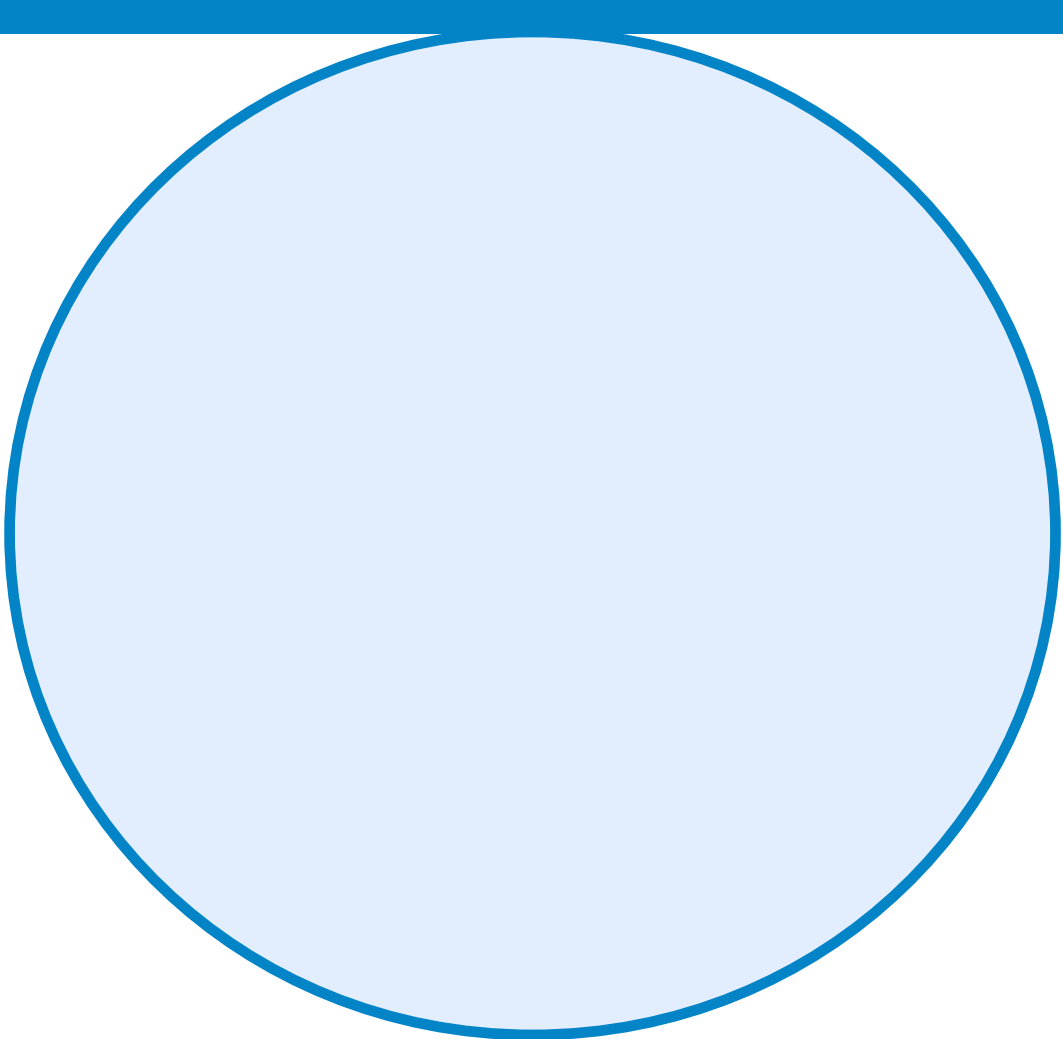
EXECUTION LOGS

Detailed Python logging for execution tracking and failure diagnosis, the log reports were made.



GITHUB REPOSITORY

Full codebase with README, feature files, page objects, and data files version-controlled



THANK YOU

Automation Testing on BestBuy Web Application

Selenium • Pytest • BDD with Behave • Wipro Capstone Project • 2026

Team BestBuy Automation | Wipro

